

# Beyond Task Completion: Auditing Evolving Tool Libraries in Agentic AI

Anonymous Author(s)

## Abstract

Agentic AI systems increasingly create, modify, and reuse executable tools across tasks. Standard benchmarks measure whether the agent solves the immediate task, but rarely evaluate the persistent tool library that mediates future behavior. This creates a trustworthiness gap: an agent may achieve high task completion while accumulating brittle, redundant, weakly reusable, or regression-inducing tools. We introduce EVOLVETOOL-BENCH, a diagnostic framework for evaluating evolving tool libraries as auditable software artifacts. The benchmark organizes tasks into seed, gap, variant, composition, regression, and adversarial phases, and reports decomposed metrics for verified task completion, tool conformance, correct reuse, incorrect reuse, redundancy, composition, and regression. We evaluate six tool-creation protocols and a no-evolution baseline across three code-execution domains, 99 tasks, and two Claude models. Three findings motivate the framework. First, task completion alone is insufficient: TC clusters in a 63–72% band while library health spans a much wider range, with library health and task completion empirically orthogonal at session granularity ( $\rho = -0.03$ , 144 sessions). Second, raw reuse is ambiguous: EVOLVETOOL-BENCH decomposes reuse into correct and incorrect components, exposing a “reuse paradox” where the highest-reuse system propagates defects 38.9% of the time. Third, self-generated validation misses specification gaps: per-tool correctness under capability-aligned hidden tests is near zero (0–6%) despite TC > 60%, showing that tool-promotion decisions require external conformance signals. EVOLVETOOL-BENCH emits per-task audit traces and preserved tool artifacts, enabling researchers to inspect *why* systems succeed or fail and to incorporate library-level diagnostics into future agent evaluations. The contribution is a diagnostic methodology and a set of reusable evaluation practices, not a leaderboard.

## 1 Introduction

LLM agents increasingly generate executable software artifacts at runtime (Python functions, API clients, data parsers) and accumulate them in persistent libraries variously called *tools*, *skills*, or *actions*. This capability appears across diverse systems: VOYAGER [Wang et al.(2023)] synthesizes code skills in Minecraft, CREATOR [Qian et al.(2023)] uses tool creation as a reasoning strategy, LLMs as Tool Makers [Cai et al.(2023)] separates tool-making from tool-using, EvoSkill [Alzubi et al.(2026)] evolves structured skill folders via failure analysis, ToolMaker [Wölflein et al.(2025)] turns GitHub repos into LLM-callable tools via closed-loop self-correction, and CoEvoSkills [Zhang et al.(2026)] pairs skill generation with adversarial verification. As reusable procedural knowledge is increasingly packaged, shared, and composed across agents, the quality of these generated artifacts becomes a first-order concern.

From a software engineering perspective, each of these systems is an *automated code generator* whose output accumulates into a

growing codebase. Yet they are evaluated exclusively on task success, analogous to judging a software engineer solely by whether their code runs, ignoring maintainability, duplication, test coverage, regression, and technical debt. In practice, skill libraries can pass downstream benchmarks while accumulating redundant functions, failing on edge cases, breaking previously working capabilities, and never reusing existing code. Recent work confirms the risk: SkillsBench [Li et al.(2026)] found that self-generated skills *hurt* agent performance by 16.2 points in roughly one-third of tasks. **Existing benchmarks do not isolate *why* this happens:** whether the degradation stems from incorrect implementations, redundant code, regression, or failed reuse.

We present EVOLVETOOL-BENCH, a diagnostic framework that treats LLM-generated tool and skill libraries as software artifacts subject to quality evaluation. The framework is system-agnostic: any tool-generating or skill-generating agent can be evaluated against the same tasks and metrics. Our contributions are a diagnostic methodology and a set of reusable evaluation practices:

- (1) **Structured diagnostic sessions.** We organize evaluation into sessions of 11 tasks with known dependency relationships (seed, gap, variant, compose, regress, adversarial). This structure enables causal analysis — researchers can determine not just whether a system failed, but *at which phase* and *for what reason* (Figure 1).
- (2) **Decomposed artifact-level metrics.** We define a per-tool Tool Quality Score (TQS) covering correctness, robustness, generality, and code quality, and six library-level metrics — correct reuse, incorrect reuse, redundancy, utilization, composition, and regression stability — forming the EvolveTool Score (ETS). We deliberately exclude safety from the composite pending a rigorous implementation, and document this choice transparently.
- (3) **Correct-vs-incorrect reuse decomposition.** We show that raw reuse rate is an ambiguous signal: a system with high reuse can propagate defects at high rate. EVOLVETOOL-BENCH separates correct reuse (reuse that leads to task success) from incorrect reuse (reuse of a defective tool), providing an actionable signal for library-management policy.
- (4) **Capability-aligned hidden tests and auditable traces.** Tool quality is evaluated using hidden tests aligned to the capability each tool was created for — not a single shared test pool. Every run emits per-task audit traces preserving agent output, tool artifacts, verification results, and LLM call counts, enabling downstream inspection and reproduction.

## 2 Related Work

*Quality of LLM-Generated Code.* Tool-Genesis [Xia et al.(2026)] evaluates one-shot tool creation with surface compliance, schema-F1, and unit test pass rates, all per-artifact metrics with no library-level evaluation. SkillsBench [Li et al.(2026)] is the closest work in motivation: it evaluates whether curated skills improve agent task success and finds that self-generated skills *degrade* performance by 16.2 points in ~33% of tasks, a result that directly motivates our work. However, SkillsBench uses static, pre-curated skill sets and measures only the downstream effect (did performance improve?), not the *cause* (is the skill buggy? redundant? regression-inducing?). Our benchmark complements SkillsBench by providing the diagnostic layer: per-tool quality scores, library-level metrics, and structured sessions that isolate *why* a skill library helps or hurts. Our “reuse paradox” (38.9% incorrect reuse in Code-Evol/Sonnet over 4 seeds; §5.4) offers one explanation for SkillsBench’s finding: agents reuse skills they correctly identify as relevant but that contain implementation defects. NofunEval [Singhal et al.(2024)] evaluates non-functional aspects of LLM-generated code (efficiency, maintainability) on isolated samples; our work differs by evaluating non-functional properties at the library level, across a growing codebase with reuse and regression. CRAFT [Yuan et al.(2023)] retrieves from specialized toolsets but does not measure the quality of the toolset itself. Berlot-Attwell et al. [Berlot-Attwell et al.(2026)] evaluate library learning quality in program synthesis, measuring whether learned abstractions generalize. Their work is similar in spirit to ours, though focused on program synthesis rather than agent skill evolution. None of these works measure redundancy, regression, or reuse across a growing codebase.

*Self-Evolving Agent Frameworks.* VOYAGER [Wang et al.(2023)] demonstrates lifelong skill acquisition in Minecraft, evaluated by exploration metrics. EvoSkill [Alzubi et al.(2026)] evolves textual skill folders through failure-driven feedback, evaluated by downstream accuracy. ToolMaker [Wölflin et al.(2025)] introduces a closed-loop self-correction protocol with structured error feedback, evaluated by per-tool unit-test pass rates. Each generates software artifacts that accumulate into a skill library, but none evaluate the quality of the resulting library, a gap our benchmark fills.

*Software Engineering Benchmarks for Agents.* ToolBench [Qin et al.(2023)], AgentBench [Liu et al.(2024)], and ToolComp [Nath et al.(2025)] evaluate agents using pre-defined tool sets. Vending-Bench [Backlund and Petersson(2025)] tests long-horizon coherence with fixed tools. None evaluate agents that *generate* their own tools, and none apply software quality metrics to the generated artifacts.

*Program-Synthesis Library Learning.* A separate lineage of work, originating in program synthesis rather than LLM agents, has long measured the quality of *learned libraries*. DreamCoder [Ellis et al.(2021)] bootstraps inductive program synthesis by alternately compressing solved problems into a library and solving harder ones from it, evaluating the library by compression ratio and downstream solve rate. Stitch [Bowers et al.(2023)] provides a faster top-down algorithm for the same abstraction-discovery problem; babble [Cao et al.(2023)] uses e-graphs and anti-unification for the same goal. AbstractBeam [Zenkner et al.(2024)]

brings library learning to bottom-up bottom-up synthesis. Leroy [Bellur et al.(2024)] extends library learning to imperative languages. Berlot-Attwell et al. [Berlot-Attwell et al.(2026)] recently argued that much purported library learning in LLM-based program synthesis fails when one accounts properly for compute and behaviour. Our benchmark differs in two ways: (i) the artifacts under test are *LLM-generated tools for natural-language-described tasks*, not abstractions extracted from solved programs, and (ii) we measure correctness against *hidden tests* alongside compression-style reuse, so a library that achieves high reuse with defective tools (our “reuse paradox”) is not awarded the same score as one with correct reuse. We see EvolveTool-Bench as complementary to this lineage rather than competing with it.

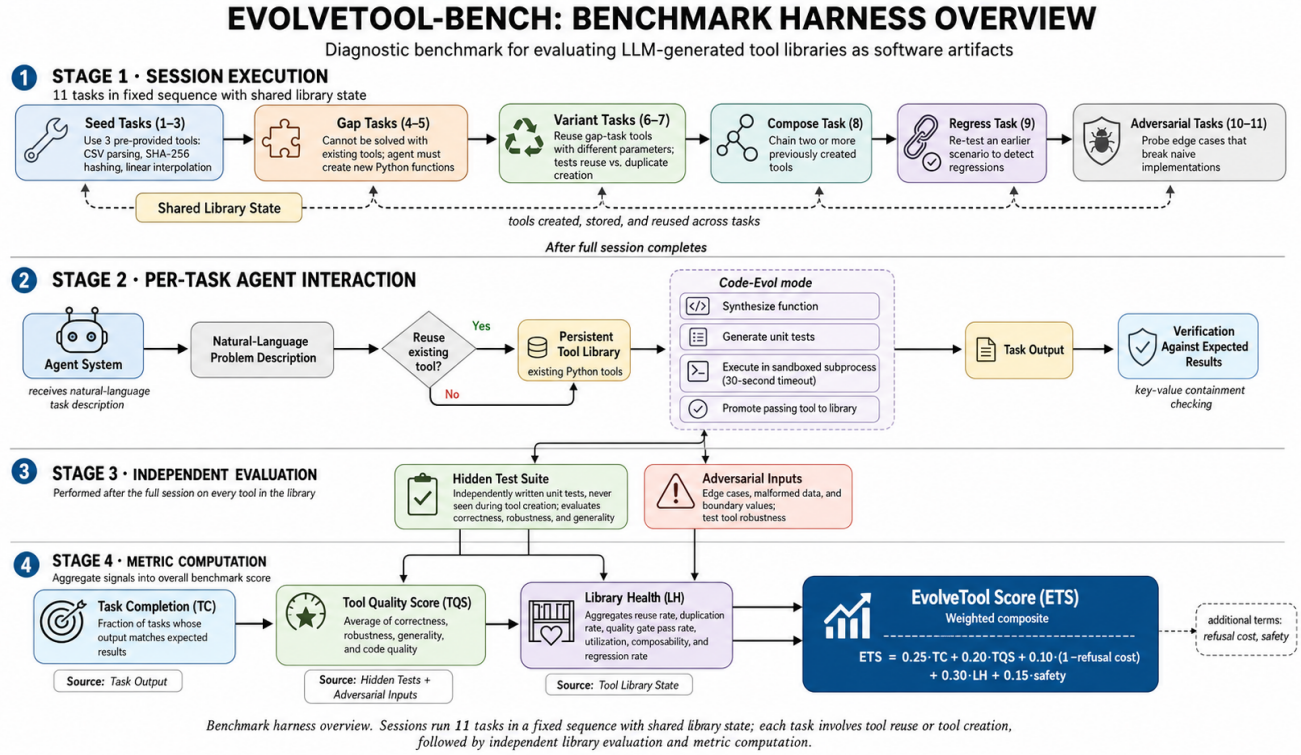
*Concurrent Tool/Skill Benchmarks.* Two contemporaneous benchmarks deserve direct comparison. The ToolMaker pipeline of Wölflin et al. [Wölflin et al.(2025)] introduces TM-Bench, which evaluates whether an agent can convert a research paper into a runnable tool that an LLM-as-judge can verify. CoEvoSkills [Zhang et al.(2026)] pairs skill generation with adversarial verification and reports skill-level pass rates. Both focus on per-tool correctness; neither evaluates redundancy, regression, utilization, or composability of the resulting *library*. Tool-Genesis [Xia et al.(2026)] is the closest in spirit (it evaluates schema-F1 and unit-test pass rates of generated tools), but again at the per-tool level. EvolveTool-Bench’s contribution is orthogonal: we evaluate the library as a software artifact, and any of these systems could be plugged into our harness as an additional configuration.

*Gap.* To our knowledge, existing benchmarks do not *jointly* measure redundancy, reuse, regression, composition, and code quality of self-evolved skill libraries as software artifacts *at the library level*. SkillsBench shows that skills can hurt; we provide the diagnostic framework to explain *why*. EVOLVETOOL-BENCH fills this gap with a system-agnostic evaluation framework.

## 3 Benchmark Design

We first define key terms used throughout. A *session* is a fixed sequence of 11 tasks executed in order with shared library state. A *task* is a single problem instance with a natural-language description, expected output format, and verification criteria. A *dependency* is a directed relationship where a later task is designed to be solvable using artifacts created during an earlier task. Tasks are designed to require Python code execution and tool use rather than text-only reasoning; the domains (§3.4) use proprietary formats and computations chosen so that pattern-matching against training data is insufficient.

The benchmark evaluates two artifact granularities. A **tool** is a single Python function with documentation (the unit most existing systems emit). A **skill** is a richer multi-file bundle: a callable paired with a SKILL.md description, auto-generated unit tests, and a metadata.json manifest (the format used by Anthropic Claude Skills and by CoEvoSkills [Zhang et al.(2026)]). Most baselines we evaluate synthesise tools; a skill-format track (supplement) adds bundles in which the expected artifact is a skill. We use *artifact* as the umbrella term.



**Figure 1: Benchmark harness overview.** Top: each session runs 11 tasks in a fixed sequence; annotations show what each task type tests. Middle: the agent system receives each task as natural language, creates or reuses Python tools in a persistent library, and produces output. After the session, hidden tests and adversarial inputs independently evaluate the library. Bottom: four metrics are computed: task completion (TC) from outputs, per-tool quality (TQS) from hidden tests, library health (LH) from library state, and the composite EvolveTool Score (ETS).

Figure 1 shows the benchmark harness. Each session runs 11 tasks in a fixed order; the agent system receives each task as a natural-language description, uses or creates Python tools in a shared library, and produces output verified against expected results.

### 3.1 Task Session Structure

Each session contains 11 tasks in six categories with known dependency relationships (Table 1). This structure mirrors software engineering workflows: seed tasks test existing code, gap tasks require new implementation, variant tasks test code reuse, compose tasks test function composition, regress tasks detect regressions, and adversarial tasks probe edge cases.

The task ordering within each session is deliberate. Seed tasks (1–3) establish baseline functionality using pre-provided tools. Gap tasks (4–5) present problems that *cannot* be solved with existing tools, forcing the system to create new ones. Variant tasks (6–7) present problems solvable by the tools created in gap tasks with different parameters, the key test of whether the system recognizes a reuse opportunity or wastefully creates a duplicate. The compose task (8) requires chaining two or more previously created tools. The regress task (9) re-tests an earlier scenario to detect regressions

| Type        | N | SE Analog                |
|-------------|---|--------------------------|
| Seed        | 3 | Use existing library     |
| Gap         | 2 | Implement new feature    |
| Variant     | 2 | Reuse vs. rewrite        |
| Compose     | 1 | Function composition     |
| Regress     | 1 | Regression testing       |
| Adversarial | 2 | Edge case / fuzz testing |

**Table 1: Task types and their software engineering analogs.** Each session follows a fixed sequence enabling controlled measurement of library evolution.

introduced by library growth. Adversarial tasks (10–11) probe edge cases designed to break naive implementations.

*What the regress task measures (and what it does not).* Real software regularly changes interfaces in response to evolving requirements; “different behaviour” is not always “regression.” We restrict  $\delta$  to *behaviour-preserving* regressions: each regress task replays the *exact same input/output contract* as an earlier task in the session, with no new requirements injected. The expected output is unchanged, so if the agent’s answer is now wrong, the only explanation is that library growth (a new tool, a refinement, or a wrapper) has broken a previously-working capability. This deliberately



excludes the legitimate-interface-change case. A complementary “requirement-evolution” axis, in which a later task changes the contract on purpose and asks whether the agent migrates cleanly, is an extension we leave to future work; the current  $\delta$  should be read narrowly as *regression under fixed requirements*.

### 3.2 Reuse Detection

The reuse metric measures whether the agent recognizes that an existing tool can solve a new task. Before each task, the harness records the current library ( $\mathcal{L}_{\text{before}}$ ). After the task, it identifies newly created tools ( $\mathcal{L}_{\text{new}}$ ) and checks whether any tool *used* by the agent was from the pre-existing set. Formally, for variant and regress tasks:

$$\text{reused}(t) = \exists f \in \text{tools\_used}(t) : f \in \mathcal{L}_{\text{before}} \setminus \mathcal{L}_{\text{new}} \quad (1)$$

This measures the *behavioral signal*: did the system attempt reuse? This is measured independently of whether the trajectory was correct. We further decompose reuse into *correct reuse* (reused tool AND task passed) and *incorrect reuse* (reused tool AND task failed); see Section 5.4.

### 3.3 Composition Tasks

Each compose task has an explicit `composes_tasks` field referencing prior task IDs whose tools must be chained. For example, an API orchestration compose task references [“gap\_1”, “gap\_2”]: the agent must chain the authentication tool (from gap\_1) with the pagination tool (from gap\_2) to fetch and aggregate all users.

We do *not* guarantee that the prerequisite tools exist in the library. If the system failed to create them during earlier gap tasks, the compose task becomes harder; it must solve everything from scratch. This is realistic: a fragile library makes composition harder, and the metric captures this cascading effect.

### 3.4 Domains

We design three domains with tasks that require *actual code execution* and cannot be solved by pattern-matching against training data.

**Domain A: Data Transformation (5 sessions, 55 tasks).** Five proprietary binary formats designed to have no overlap with LLM training data: custom binary serialization with CRC checksums (ABR), run-length encoded matrices (RLE), a schema validation language (VDL), binary log format with variable-length entries (QLOG), and nested serialization with parity-based error correction (TPACK).

**Domain B: API Orchestration (1 session, 11 tasks).** A mock HTTP server implementing HMAC-timestamp authentication and XOR-encrypted cursor pagination. Tasks require the agent to authenticate, paginate through results, handle cursor decryption, and compose these capabilities.

**Domain C: Numerical Computation (3 sessions, 33 tasks).** Custom scientific computing formats for nonlinear curve fitting (ARCFIT), signal processing with FFT-based spectral analysis (ARCSIG), and constrained optimization with boundary analysis (ARCOPT).

### 3.5 Running Example: API Orchestration

A full walkthrough is provided in the supplemental material; we summarise here. Seed tasks (1–3) exercise a pre-loaded `http_get` tool. Gap task 4 demands HMAC-SHA256 signing of `timestamp:path`: Code-Evol detects the failure, synthesises `compute_hmac(secret, message)`, generates 5 unit tests, validates them in a sandbox, and promotes; No-Evolution and Strategy-Only solve inline without producing a reusable artifact. Variant task 6 re-uses the auth pattern on a different endpoint — the diagnostic event for  $p_r$  and the source of the *correct reuse* vs. *incorrect reuse* split. Compose task 8 chains the HMAC tool with a pagination tool synthesised in gap 5. Regress 9 re-runs an earlier scenario to detect breakage. After all 11 tasks the harness computes TC, TQS, LH, and ETS and records which tools were reused, duplicated, or caused regressions.

### 3.6 Hidden Test Suite

The hidden tests are the only mechanism by which our TQS correctness term is independent of the system under evaluation; we document them explicitly. For every gap task we author (i) extra input/output pairs that exercise the same specification with different data, and (ii) adversarial inputs that the tool must not crash on. The benchmark contains 18 gap tasks (9 sessions  $\times$  2 gaps), 92 hidden input/output pairs, and 105 adversarial inputs — mean 5.1 correctness probes and 5.8 robustness probes per gap task (Data Transform 5.2/5.9, Numerical 5.0/6.0, API Orchestration 5.0/5.0). Counts are re-generated by `scripts/audit_tasks.py --check-hidden-tests`. The tests were written by the paper authors — “hidden from the system” is not the same as “independent of the authors” — but with two mitigations: cases were authored from the format spec (not from any reference implementation) before any system was evaluated, and each hidden input probes a different regime than the prompt example so a memorising tool still fails. With  $\sim 5$  cases per gap task, per-tool  $C$  still carries a non-trivial Bernoulli SE; we read correctness as a library-wide signal, not a per-tool ranking. The  $\sim 18$ -point library-health spread we headline is supported by the 99 task outcomes per configuration (plus multi-seed runs), not by per-tool test counts.

### 3.7 Software Quality Metrics

**Per-Tool: Tool Quality Score (TQS).** Each synthesized tool is evaluated on four dimensions:

- **Correctness (C):** Proportion of hidden unit tests passing. These tests are independently written, *not* the same tests the system generates during evolution.
- **Robustness (R):** Proportion of adversarial inputs handled without crashing.
- **Generality (G):** Proportion of held-out inputs producing correct output.
- **Code Quality (Q):** Static analysis score: docstring, type annotations, error handling, length ( $< 50$  lines), valid syntax.

$$\text{TQS}(t) = \frac{1}{4} (C(t) + R(t) + G(t) + Q(t)) \quad (2)$$

**Per-Library: Library Health (LH).** Six metrics capture library-level software quality:

$$\rho_r = \frac{|\{t \in \mathcal{T}_{v,r} : \text{reused}\}|}{|\mathcal{T}_{v,r}|} \quad (\text{Code Reuse}) \quad (3)$$

$$\rho_d = \frac{|\{(t_i, t_j) : \text{equiv}\}|}{\binom{|\mathcal{L}|}{2}} \quad (\text{Duplication}) \quad (4)$$

$$\pi = \frac{|\{t \in \mathcal{L} : \text{TQS} \geq 0.5\}|}{|\mathcal{L}|} \quad (\text{Quality Gate}) \quad (5)$$

$$\eta = \frac{|\mathcal{L}_{\text{used}}|}{|\mathcal{L}|} \quad (\text{Utilization}) \quad (6)$$

$$\gamma = \frac{|\{t \in \mathcal{T}_c : \text{passed}\}|}{|\mathcal{T}_c|} \quad (\text{Composability}) \quad (7)$$

$$\delta = \frac{|\{t \in \mathcal{T}_{\text{reg}} : \text{failed}\}|}{|\mathcal{T}_{\text{reg}}|} \quad (\text{Regression}) \quad (8)$$

$$\text{LH} = \frac{1}{6}(\rho_r + (1 - \rho_d) + \pi + \eta + \gamma + (1 - \delta)) \quad (9)$$

In plain language:  $\rho_r$  captures whether the agent recognizes existing tools when faced with similar tasks;  $\rho_d$  penalizes libraries containing functionally equivalent duplicates (detected by comparing tool outputs on a shared input set);  $\pi$  is the fraction of tools meeting the  $\text{TQS} \geq 0.5$  quality threshold;  $\eta$  measures whether created tools are actually invoked after creation;  $\gamma$  is the success rate on tasks requiring multi-tool composition; and  $\delta$  tracks whether library growth breaks previously passing behavior.

*Edge cases: empty and seed-only libraries.* Three of the four systems we evaluate (No-Evolution, Strategy-Only, and most One-Shot runs) end sessions with no newly synthesized tools, so the per-tool denominators above can be zero. We handle this explicitly:

- **Reuse**  $\rho_r$  is defined over variant and regress tasks (denominator  $|\mathcal{T}_{v,r}|$  is non-zero by session construction). Crucially, the three pre-loaded *seed tools* (parse\_csv, sha256\_hash, linear\_interpolate) count as members of  $\mathcal{L}_{\text{before}}$ . A No-Evolution agent that successfully reuses a seed tool on a variant task gets credit, which is why its reuse rate is 48.1%, not 0. This is the design choice that allows fair comparison: reuse measures the *behaviour* (did the agent re-invoke a known tool?) regardless of whether the tool was hand-written or self-evolved.
- **Duplication**  $\rho_d$ , **quality gate**  $\pi$ , **utilization**  $\eta$ , and **mean TQS**  $\overline{\text{TQS}}$  are defined over the *synthesized* library  $\mathcal{L} \setminus \mathcal{L}_{\text{seed}}$ . When this set is empty, we report these metrics as 1 for the “inverse” terms ( $1 - \rho_d = 1$ , no duplicates to penalize) and as 0 for the positive terms ( $\pi = \eta = \overline{\text{TQS}} = 0$ , nothing to credit). This is conservative: a system that creates nothing earns no quality credit but also incurs no quality penalty.
- **Composability**  $\gamma$  and **regression**  $\delta$  are defined over compose and regress tasks respectively (non-empty by construction) and are computed from task outcomes regardless of whether tools were created.

We deliberately *exclude* a safety dimension from the composite. An earlier version assigned a safety score that defaulted to 1.0 whenever no dangerous operations were statically detected; because this default fired for essentially every tool, it added a near-constant

offset that inflated all scores without discriminating between systems. Rather than report a metric that does no diagnostic work, we drop it from ETS and recommend that safety be assessed with a dedicated static/dynamic analyzer external to the composite (§6). This convention is what allows No-Evolution and Strategy-Only to receive non-trivial LH scores (0.41 and 0.41) despite synthesising zero tools: their reuse, composition, and regression numbers are real measurements, and the per-tool sub-metrics zero out cleanly. The headline LH gap to Code-Evol therefore reflects *added* quality from synthesis, not arithmetic artefacts of empty denominators.

*Composite: EvolveTool Score (ETS).* The composite score combines task completion (TC), mean tool quality ( $\overline{\text{TQS}}$ ), *refinement cost* (RC, an LLM-call-budget penalty defined below), and library health (LH):

$$\text{ETS} = 0.30 \cdot \text{TC} + 0.20 \cdot \overline{\text{TQS}} + 0.10 \cdot \max(0, 1 - \text{RC}) + 0.40 \cdot \text{LH} \quad (10)$$

where  $\text{RC} = N_{\text{LLM}} / (10 |T|)$  is the number of LLM calls made during the session normalised by ten calls per task. The constant ten was chosen empirically as the rough budget a non-evolving agent uses; systems that exceed this budget pay a linear penalty. The four weights sum to one; the previously included safety term is removed (see above).

Library health receives the largest weight (0.40) reflecting the benchmark’s thesis that library quality matters beyond task completion. LH is internally unweighted (equal contribution from all six sub-metrics) because no prior work provides empirical guidance for sub-metric importance; we leave sub-metric weighting to future work. A weight-sensitivity analysis (supplement) shows the ETS ordering is stable under weight perturbations: CREATOR-style ranks first under all five weight schemes tested — including an LH=0 stress test in which the metric the benchmark advocates for is given zero weight — consistent with the headline results (Table 2). The ranking is therefore not an artifact of the chosen weights. We nonetheless treat ETS as a dashboard summary, not the central evidence; the primary claims rest on the decomposed metrics with confidence intervals (Table 3).

## 4 Experimental Setup

To validate the benchmark’s discriminative power, we evaluate five core systems representing different points on the tool-evolution spectrum (two further adapted baselines are described afterward):

- (1) **No-Evolution:** The agent receives 3 pre-written seed tools (CSV parsing, SHA-256 hashing, and linear interpolation) and solves all tasks using inline reasoning with these fixed tools. The library is never modified. This isolates the LLM’s baseline problem-solving ability from any tool-creation mechanism.
- (2) **One-Shot:** When the agent encounters a task it cannot solve with existing tools, it generates a single Python function in one LLM call and adds it to the library immediately. No test cases are generated, no sandbox validation is performed, and no iterative refinement occurs. If the function is buggy, it persists in the library and may be reused on later tasks.
- (3) **Strategy-Only:** A strategy-level baseline inspired by EvoSkill [Alzubi et al.(2026)], using their published Skill

Proposer and Prompt Generator templates. Instead of generating executable code, the system maintains textual strategy folders containing natural-language descriptions like “To authenticate: compute HMAC-SHA256 of timestamp+path with the API secret.” These strategies are not parsed or executed; they are inserted as natural-language hints into the agent’s prompt. This is *not* a faithful reimplement of EvoSkill’s full system (which uses different infrastructure); it characterizes what strategy-level evolution alone achieves. See Limitations.

- (4) **Code-Evol**: Code-level evolution with sandbox validation. When a task fails, the system: (a) analyzes failure trajectories to detect capability gaps, (b) synthesizes a Python function with type hints and docstrings, (c) generates 5–7 unit tests for the function, (d) executes both in a subprocess sandbox with a 30-second timeout, and (e) promotes passing tools to the library. An LLM-as-judge evaluates each trajectory on a 0.0–1.0 scale; trajectories scoring below 0.5 trigger evolution. Failed tools enter a refinement loop (up to 3 attempts).
- (5) **ToolMaker-style**: A faithful adaptation of the ToolMaker protocol [Wölflein et al.(2025)] to our session format. When a task fails, the agent synthesizes a function plus 3–5 auto-tests in a single call; the auto-tests run in a sandbox; if any fails, the captured stdout/stderr (with `traceback.format_exc()`) feeds a *diagnose* call that produces a structured diagnosis-plus-plan, followed by a *rewrite* call. The loop iterates up to 3 times. Distinct from Code-Evol in that the error signal is test-execution output rather than trajectory-level LLM-judge scoring, and the *diagnose/rewrite* split mirrors ToolMaker’s two-stage prompt structure. We adapt the protocol rather than literally porting the repo (their codebase is OpenAI-only, Docker-based, and expects a GitHub repo as input).

These systems span a spectrum from no tool creation through unvalidated creation, strategy-level adaptation, validated code synthesis with LLM-judge guidance, and structured-error self-correction. We additionally evaluate **CREATOR-style** [Qian et al.(2023)] (decision/execution-split tool creation) and **ToolCoder-style** (code-generation-centric tool synthesis) under the same harness. In total this gives six tool-creation protocols (One-Shot, Strategy-Only, Code-Evol, ToolMaker-style, CREATOR-style, ToolCoder-style) plus the No-Evolution baseline; the five Sonnet systems run at  $n=4$  seeds are the *main* configurations, and ToolCoder-style/ToolMaker-style/Haiku/semantic-Q rows are single-seed appendix variants.

**Baseline fairness.** Several baselines (Strategy-Only, CREATOR-style, ToolCoder-style, ToolMaker-style) are protocol *adaptations* rather than native ports, because the original systems assume incompatible infrastructure (OpenAI-only, Docker, GitHub-repo inputs, or pre-defined reference tests). The unifying principle is that every system sees the identical task stream, seed library, model, and verification function; only the tool-creation *protocol* differs. Where an adaptation could disadvantage a baseline — notably ToolMaker-style, whose *diagnose/rewrite* loop is gated by reference tests our open tasks do not provide — we flag it explicitly rather than report the number without context. A per-baseline fairness ledger

(native setting, what was preserved, what was adapted, and why) is provided in the supplement.

**Models.** Two Claude models via Amazon Bedrock: Sonnet 4 and Haiku 4.5. All systems run on Sonnet; Code-Evol and No-Evolution also run on Haiku. Each main Sonnet configuration runs at  $n=4$  seeds. Initial GPT-4o runs were degenerate (TC=48.5%, zero tools across all protocols, harness  $\times$  function-calling interaction we have not root-caused); we omit them and revisit in v3.

**Scale.** 9 sessions  $\times$  11 tasks = 99 tasks per configuration; 25 individual runs in total (20 Sonnet across 5 baselines  $\times$  4 seeds, plus 3 single-seed Sonnet variants and 2 Haiku reference runs).

**Implementation details.** All systems implemented in Python 3.11; Claude models accessed via Amazon Bedrock. All LLM calls use default temperature (1.0) and top-p (1.0) with `max_tokens=4096` for agent responses and `max_tokens=10` for the LLM-as-judge score. Code-Evol’s sandbox executes synthesized tools in isolated subprocesses with a 30-second timeout and no network access. Each session starts with a clean library containing only the 3 seed tools; library state does not carry across sessions.

**Verification.** Task outputs are checked against a per-task expected mapping using lenient key-value containment: if 80%+ of expected key-value pairs appear in the agent’s output, the task passes. The 80% threshold was calibrated via pilot runs to accommodate format variation (e.g., key ordering, whitespace) while rejecting substantively wrong outputs. The harness fails *closed*: a task with no expected mapping or custom `verify` predicate is scored as a failure rather than passed on a length heuristic. We audit verifier coverage with a released script (`scripts/audit_tasks.py`); at present 51/99 tasks carry deterministic verifiers, with the API-orchestration session the largest gap. The TC levels reported here were computed under the original lenient regime, which could credit a task lacking an expected mapping; under the current fail-closed policy such tasks score as failures, so strict verified TC would be no higher. We therefore treat the cross-system TC *ranking and clustering* — not the absolute level — as the robust result, and flag full deterministic verification plus a re-run of affected configurations as the highest-priority hardening item (L9).

## 5 Results

### 5.1 The Benchmark Reveals Hidden Quality Differences

Table 2 and Figure 2 present two findings.

**Task completion clusters; library health separates.** On Sonnet, six of seven configurations land in a narrow 63–68% TC band, yet their library health spans 36.4–54.8% — an  $\sim 18$ -point spread invisible to TC-only evaluation. No-Evolution (TC= 66.7, LH= 39.7) creates no tools and leaves the library unchanged; One-Shot (TC= 67.4, LH= 38.6) creates three untested tools that *degrade* it. A TC-only benchmark would call these equivalent; they are not.

**Per-tool correctness is universally low under hidden testing.** Under the hidden test suite (5+ per gap task, authored from the format specification before any system was evaluated), per-tool correctness across every tool-creating system sits in a 0–6% range despite

| System                     | Model  | $n$ | ETS $\uparrow$                    | TC (%)         | Tools | TQS   | Reuse (%)      | LH (%)                         |
|----------------------------|--------|-----|-----------------------------------|----------------|-------|-------|----------------|--------------------------------|
| No-Evol                    | Sonnet | 4   | 0.439 $\pm$ 0.007                 | 67.6 $\pm$ 0.7 | 0     | —     | 46.3 $\pm$ 1.9 | 39.7 $\pm$ 1.7                 |
| One-Shot                   | Sonnet | 4   | 0.443 $\pm$ 0.010                 | 67.4 $\pm$ 1.1 | 3     | 0.044 | 44.4 $\pm$ 2.6 | 38.6 $\pm$ 2.4                 |
| Strategy-Only              | Sonnet | 4   | 0.442 $\pm$ 0.006                 | 66.9 $\pm$ 0.6 | 0     | —     | 50.0 $\pm$ 4.1 | 41.2 $\pm$ 1.6                 |
| CREATOR-style              | Sonnet | 4   | <b>0.576<math>\pm</math>0.030</b> | 67.7 $\pm$ 3.1 | 23.5  | 0.460 | 66.7 $\pm$ 7.9 | <b>54.8<math>\pm</math>5.6</b> |
| ToolCoder-style            | Sonnet | 1   | 0.507                             | 54.5           | 17    | 0.449 | 44.4           | 49.2                           |
| ToolMaker-style $^\dagger$ | Sonnet | 1   | 0.413                             | 63.6           | 0     | —     | 40.7           | 36.4                           |
| Code-Evol                  | Sonnet | 4   | 0.543 $\pm$ 0.014                 | 64.9 $\pm$ 0.9 | 21    | 0.354 | 72.2 $\pm$ 4.1 | 51.6 $\pm$ 2.9                 |
| Code-Evol (sem. Q)         | Sonnet | 1   | 0.561                             | 62.6           | 15    | 0.432 | 63.0           | 52.8                           |
| No-Evol                    | Haiku  | 1   | 0.439                             | 66.7           | 0     | —     | 51.9           | 40.1                           |
| Code-Evol                  | Haiku  | 1   | 0.546                             | 65.2           | 13    | 0.353 | 66.7           | 51.2                           |

**Table 2: Full results across systems, models, and seeds.  $n$  is the seed count. Multi-seed runs report mean $\pm$ std; single-seed runs are point estimates. *Code-Evol (sem. Q)* reruns Code-Evol with the semantic  $Q$  refinement; not directly comparable to surface- $Q$  rows.  $^\dagger$ ToolMaker-style created zero tools because our open-task harness lacks pre-defined reference tests to gate the diagnose-and-fix loop (Sec. 6).**

**Table 3: Library-health contrasts (Sonnet,  $n=4$  seeds  $\times$  9 sessions). Paired hierarchical bootstrap over (seed, session), 10k resamples; BH-adjusted bootstrap  $p$ . Generated by scripts/stats\_report.py.**

| Contrast (LH)                 | $\Delta$ | 95% CI      | $p_{\text{adj}}$ |
|-------------------------------|----------|-------------|------------------|
| Code-Evol – No-Evol           | 12.0*    | [6.8, 17.0] | 0.000            |
| CREATOR-style – No-Evol       | 15.2*    | [8.0, 23.4] | 0.000            |
| Code-Evol – Strategy-Only     | 10.4*    | [6.4, 14.7] | 0.000            |
| CREATOR-style – Strategy-Only | 13.6*    | [7.5, 21.3] | 0.000            |

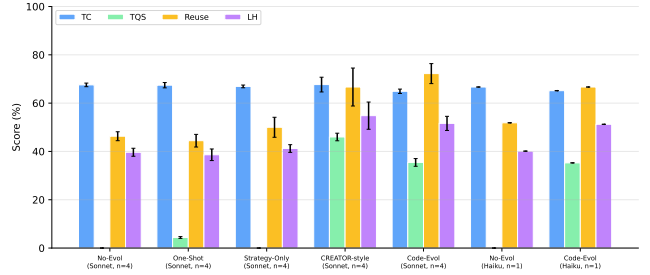
\*95% CI excludes 0. LH in percentage points.

TC>60%. The 63–72% task-completion numbers ride on a library of tools that mostly fail when held to capability-aligned hidden-test standards. Implications in §6.

*Library-management protocols raise LH significantly (primary claim).* We test the four pre-registered library-health contrasts with a paired hierarchical bootstrap over (seed, session) pairs (10k resamples; BH-adjusted), regenerated by scripts/stats\_report.py (Table 3). All four confidence intervals exclude zero: Code-Evol and CREATOR-style improve LH over both the No-Evolution and Strategy-Only baselines by 10–15 percentage points ( $p_{\text{adj}} < 0.001$ ). This is the benchmark’s strongest statistically supported finding, and it holds even though those same systems are statistically indistinguishable from the baselines on TC.

*Raw reuse and correct reuse diverge (primary claim).* Reuse precision (correct reuse / raw reuse) separates productive abstraction from defect propagation. Code-Evol has the *highest* raw reuse (72.2%) but the *lowest* precision (20.2%, 95% CI [9.6, 32.3]): most of its reuse propagates a defective tool. CREATOR-style (raw 66.7%, precision 39.6% [24.5, 55.2]) and Strategy-Only (raw 50.0%, precision 39.9% [22.0, 57.1]) reuse less but more correctly. Reporting raw reuse alone is therefore misleading; the correct/incorrect decomposition is the reusable measurement contribution.

*On CREATOR-style vs. Code-Evol.* CREATOR-style attains the highest point-estimate ETS and LH, but at  $n=4$  seeds the



**Figure 2: System comparison across four metrics (%). TC clusters all systems together (63–68%); library health separates them (38–52%). This illustrates the benchmark’s discriminative power beyond task completion.**

CREATOR–Code-Evol LH difference is not separable from noise; we treat the CREATOR-vs-Code-Evol ordering as *descriptive* and do not claim one protocol dominates the other. The supported claim is the contrast against the no-management baselines, not the ranking among management protocols.

*Domain profiles.* Tool-evolution effectiveness varies by problem type (full per-domain table in the supplement): for Code-Evol, API orchestration yields the highest library health (77.8%, single session), data transformation creates the most tools (14 on Sonnet) but at lower quality (47.6% LH, complex binary parsers failing held-out tests), and numerical computation is consistently difficult across models (~48–50% LH). This domain-level variation is itself invisible to aggregate TC.

## 5.2 Per-Dimension TQS: Where Quality Breaks Down

The per-dimension TQS breakdown (Table 4) pinpoints *where* tool quality fails:

*Code quality  $Q$  ranges 0.67–0.92.* LLMs consistently produce well-structured code. The semantic- $Q$  refinement (cyclomatic complexity, MI, control-flow checks) broadens the  $Q$  discrimination range



| System (Sonnet)               | $n_{\text{tools}}$ | C   | R   | G   | Q   | TQS |
|-------------------------------|--------------------|-----|-----|-----|-----|-----|
| One-Shot ( $n=3$ seeds)       | 9                  | .00 | .50 | .00 | .69 | .30 |
| ToolCoder-style               | 15                 | .00 | .52 | .10 | .92 | .38 |
| CREATOR-style ( $n=4$ seeds)  | 81                 | .01 | .50 | .06 | .73 | .33 |
| Code-Evol (semantic- $Q$ run) | 15                 | .01 | .55 | .11 | .95 | .41 |

**Table 4: Per-tool TQS dimensions, averaged over all preserved tools (not session-diluted as in Table 2).  $C$  is essentially zero for every system under the capability-aligned hidden tests;  $Q$  remains high (0.69–0.95), confirming LLM-generated tools are well-structured but rarely semantically correct.**

from  $\sim 0.9$  under the surface-only metric to 0.67–0.92 under the refined one. The gap between  $Q$  and  $C$  remains partly definitional:  $Q$  captures structural quality,  $C$  captures semantic correctness.

*Correctness is universally near zero.* Capability-aligned hidden tests return per-tool correctness in the 0–6% range for every tool-creating system (per-system seed averages  $\leq 1\%$ ). Tools that pass each system’s self-generated tests still fail on diverse inputs — the prompt example is the one input on which they were exercised at task time. This is the central diagnostic finding of the benchmark.

*Robustness clusters at 0.50.* Almost every system wraps synthesised functions in try/except blocks; they avoid crashes (earning half-credit on robustness) but do not produce correct outputs (no credit on correctness). The robustness dimension as currently defined therefore does not discriminate; we treat this as a benchmark weakness to refine rather than as a property of the systems.

*Generality tracks correctness.* Tools that fail hidden tests also fail on unseen inputs, indicating fundamental misunderstanding of the format specification rather than overfitting.

*Library-level sub-metrics.* Two LH sub-metrics deserve explicit reporting. The quality gate ( $\pi$ ) shows that only 36.4% of Code-Evol/Sonnet tools and 46.2% of Code-Evol/Haiku tools pass TQS  $\geq 0.5$ ; most synthesised tools fail on correctness despite passing self-validation. The duplication metric ( $\rho_d$ ) reveals that Code-Evol/Sonnet creates decode\_base64 three times across sessions and decode\_binary\_data twice. This cross-session redundancy is invisible to per-session evaluation and confirms that library-level measurement catches failure modes that per-tool metrics miss.

### 5.3 Artifact Gallery (Appendix)

Aggregate numbers obscure the artifacts being scored. Tool preservation archives every synthesised tool’s source to disk, enabling a per-tool catalogue. We provide an extensive gallery in the supplement: representative tools from both tool-creating systems (Code-Evol with semantic- $Q$  and CREATOR-style), grouped by failure mode (*correct exemplar*, *fragile* with high  $Q$  but low  $C$ , *duplicate* across sessions, *trivial*), each with full signature, docstring, body excerpt, and per-dimension scores.

### 5.4 The Reuse Paradox

The benchmark reveals an unexpected interaction between reuse and task success (Table 5). Code-Evol has the highest raw reuse (72.2%) but only 15.7% of its reuse is *correct* — the agent correctly

| System (Sonnet, $n=4$ ) | Reuse | Correct | Incorrect |
|-------------------------|-------|---------|-----------|
| No-Evolution            | 46.3% | —       | —         |
| One-Shot                | 44.4% | —       | —         |
| Strategy-Only           | 50.0% | 22.2%   | 27.8%     |
| CREATOR-style           | 66.7% | 29.6%   | 37.0%     |
| ToolCoder-style*        | 44.4% | 7.4%    | 37.0%     |
| Code-Evol               | 72.2% | 15.7%   | 38.9%     |

**Table 5: Reuse paradox, decomposed. CREATOR-style breaks the paradox in a useful direction: lower raw reuse than Code-Evol but nearly 2 $\times$  the correct-reuse rate. \*ToolCoder-style  $n=1$ .**

| Task Type          | Code-Evol | No-Evol | $\Delta$ |
|--------------------|-----------|---------|----------|
| Seed (N=27)        | 100%      | 100%    | 0        |
| Gap (N=18)         | 38.9%     | 44.4%   | −5.5     |
| Variant (N=18)     | 33.3%     | 50.0%   | −16.7    |
| Compose (N=9)      | 66.7%     | 66.7%   | 0        |
| Regress (N=9)      | 33.3%     | 33.3%   | 0        |
| Adversarial (N=18) | 66.7%     | 66.7%   | 0        |

**Table 6: Task completion by type (both Sonnet, single run). Code-Evol matches No-Evolution on most task types but scores lower on gap and variant tasks. The identical compose/regress/adversarial totals are a coincidence of small N (9 tasks each); per-session results differ between systems.**

identifies reuse opportunities, but reuses tools that contain defects (a buggy-utility-function analogue). CREATOR-style breaks this pattern: lower raw reuse (66.7%) but the highest correct-reuse rate (29.6%), explaining most of its LH advantage. Practical reading: **high reuse is beneficial only when the reused tools are correct**; among our protocols, CREATOR’s simpler execute-and-rectify validation appears to commit fewer defective tools to the library in the first place.

### 5.5 Task-Type Breakdown

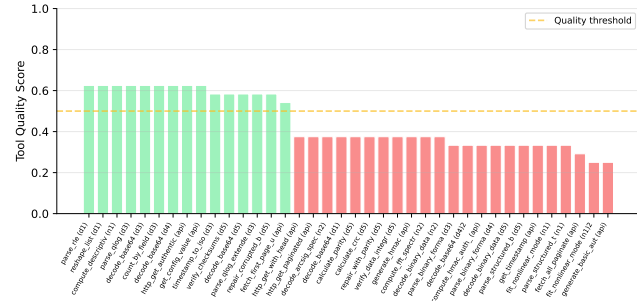
The task-type structure (Table 6) enables fine-grained analysis impossible with unstructured benchmarks:

*Tool creation has per-task costs.* Code-Evol scores lower on gap (−5.5%) and variant (−16.7%) tasks compared to No-Evolution. The evolution overhead can consume the task budget without producing a correct solution. On variant tasks, reusing a defective tool performs worse than solving inline. Code-Evol’s advantage appears at the library level (higher LH, higher reuse), not at the per-task-type level.

*Regression is universally difficult.* Both systems score 33.3% on regress tasks, indicating that library growth introduces instability regardless of evolution strategy.

*Structured sessions enable causal analysis.* Because each task type has a known purpose (gap = creation needed, variant = reuse opportunity, regress = stability test), the benchmark supports diagnostic reasoning about *why* a system succeeds or fails, not just *whether* it does.





**Figure 3: Tool Quality Score for each synthesized tool across all systems. Green = passes quality gate ( $\geq 0.5$ ). The benchmark reveals wide quality variation across the tool library.**

## 5.6 Does Library Health Predict Task Completion?

A natural follow-up question is whether LH is *predictive* of task success — that is, whether high LH measured early in a session predicts high TC later. If LH were simply a proxy for TC, the benchmark would offer no signal beyond a TC-only evaluation; if LH were strongly predictive of TC, optimizing libraries for health would also raise downstream pass rates. We test the predictive claim directly.

*Setup.* For each Sonnet session ( $n = 180$  across all 5 main configurations  $\times 4$  seeds  $\times 9$  sessions) we compute (i)  $\text{LH}_{\text{pre}}$ , the mean of the four LH sub-metrics whose value is determined by gap- and variant-task behaviour ( $\rho_r, 1 - \rho_d, \pi, \eta$ ) — the LH components that are *not* mechanically defined by compose or regress task outcomes; and (ii)  $\text{TC}_{\text{late}}$ , the mean pass rate on the compose and regress tasks of the same session. The analysis is regenerated by `scripts/stats_report.py` from the canonical manifests (10k bootstrap resamples, fixed seed).

*Result: no detectable relationship.* Across all 180 Sonnet sessions, Pearson  $r(\text{LH}_{\text{pre}}, \text{TC}_{\text{late}}) = +0.01$  (95% CI  $[-0.11, 0.13]$ ) and Spearman  $\rho = -0.03$  (95% CI  $[-0.15, 0.09]$ ). Both confidence intervals lie entirely within  $\pm 0.25$ , so a two-one-sided equivalence read supports *no meaningful monotonic association* rather than merely a failure to reject. We therefore do not detect a relationship between early library-health signals and later task completion at session granularity.

*Interpretation.* We report this conservatively: **we do not detect that LH predicts TC in our data, and the effect is bounded to be small** (equivalence within  $\pm 0.25$ ). Rather than weaken the paper’s case, this clarifies the contribution: LH and TC measure different things, so LH should be reported as a complementary diagnostic (less duplicate code, lower regression risk, higher reuse for downstream agents) rather than as a predictor of task success. Whether a sufficiently large LH gap eventually moves TC at larger scale is an open empirical question.

We also extend the benchmark to multi-file *skill bundles* (Claude Skills / CoEvoSkills [Zhang et al.(2026)] format) by wrapping each

**Table 7: Claims matrix: what the evidence supports, and where each claim sits. We headline only claims whose bootstrap CIs are decisive; protocol rankings among management systems are descriptive at  $n=4$ .**

| Claim                                                                       | Status               | Placement  |
|-----------------------------------------------------------------------------|----------------------|------------|
| Library-mgmt protocols raise LH vs. no-management baselines (CI excludes 0) | Primary              | Main       |
| Raw vs. correct reuse diverge (reuse precision)                             | Primary              | Main       |
| LH is not a TC proxy (equivalence within $\pm 0.25$ )                       | Primary              | Main       |
| Hidden-test conformance gap ( $C \approx 0$ )                               | Primary <sup>‡</sup> | Main       |
| CREATOR-style > Code-Evol                                                   | Descriptive          | Discussion |
| ToolMaker-style weak result                                                 | Protocol mismatch    | Appendix   |

<sup>‡</sup>Conditional on 51/99 deterministic verifier coverage; reported as preliminary for unverified task types.

preserved tool into a 4-file bundle and scoring structure, documentation, and metadata dimensions; Code-Evol’s documentation lead yields the largest bundle-quality gap (full table in the supplement).

*Finding 6: Audit traces make evaluation inspectable and reproducible.* Preserved per-tool artifacts, source hashes, and per-task traces allow researchers to verify the reported near-zero (0–6%) correctness numbers directly from disk, to identify exactly which specification gap caused a failure, and to reproduce any result in the artifact gallery. *Design lesson:* Agent benchmarks should publish traces, source hashes, and per-test outcomes alongside aggregate numbers.

*Finding 7: LH and TC measure different, complementary dimensions.* Across 180 sessions library health shows no detectable relationship with downstream task completion ( $\rho = -0.03$ , 95% CI  $[-0.15, 0.09]$ ; equivalence within  $\pm 0.25$ ; §5.6). LH is a *descriptive* diagnostic dimension, not a proxy for TC — the strongest empirical justification for reporting both. Whether LH becomes predictive at larger scale or under follow-up tasks against a frozen library is an open question for future work.

## 6 Discussion

*Benchmark Design Principles.* EVOLVETOOL-BENCH embodies three design principles: (1) *held-out validation*, where quality metrics are computed by tests hidden from the evaluated system (authored from the spec, not third-party); (2) *library-level measurement*, because per-tool metrics alone miss emergent properties like redundancy and regression; and (3) *structured sessions*, where known task dependencies enable causal analysis of reuse, composition, and stability.

*The task-completion / tool-correctness gap.* The central diagnostic finding of the benchmark is the gap between two numbers the field has implicitly conflated: *task completion* (does the agent produce an answer matching the expected output?) and *tool correctness* (does the synthesised callable, in isolation, produce correct outputs across the input distribution?). Both numbers are reproducible from the canonical manifests (`results_canonical/`); the tools are on disk with source hashes. What the gap reveals is that the agent’s success often comes from features other than a fully correct callable — the

prompt example is the only input the tool is exercised on at task time, and an implementation that handles that example produces a passing TC outcome even if it fails everywhere else. A single-example TC check is a much weaker correctness signal than its widespread use suggests; library-level multi-dimensional reporting (hidden conformance tests, quality gates, reuse decomposition) is needed for honest claims about what LLM-synthesised tools can be relied upon to do.

*Where do systems actually differ?* Per-tool correctness, generality, and robustness are clustered. At the library level there is more separation: CREATOR-style and Code-Evol reach LH above 50%, the non-synthesising baselines and One-Shot sit at 38–41%. The gap is driven by reuse and quality-gate components, not per-tool quality — the protocols differ chiefly in *library management* (when to create, when to reuse, what to commit), not in the correctness of any individual artifact. We read this as evidence that the next round of progress on tool-creating agents will come from better library management as much as from better synthesis.

*Implications for Self-Validation.* On our benchmark, self-generated tests are insufficient: correctness is near zero (under 0.06 per system) on hidden tests despite tools passing their own validation. While we cannot generalize this to all LLM-generated code, the pattern (LLMs generating both code and tests share the same misconceptions about a specification) suggests independent validation remains important for systems that deploy LLM-synthesized tools.

*Limitations.* We acknowledge open limitations explicitly. **(L1) Predictive validity of LH at scale.** §5.6 shows no detectable LH–TC relationship at session granularity ( $\rho = -0.03$ , 95% CI  $[-0.15, 0.09]$ , over 180 sessions); whether LH becomes predictive at larger scale or under follow-up tasks against a frozen library is open. **(L2) Baseline ecosystem.** The six protocols span a broad design space but several contemporaneous systems (Tool-Genesis, CoEvoSkills) could not be evaluated directly; our ToolMaker-style protocol, while implemented faithfully, requires reference tests our open-task harness does not provide. **(L3) Hidden tests** average 5+ per gap task and are author-written; third-party shadow tests would strengthen the independence guarantee further. **(L4) Statistical power on Haiku.** All main Sonnet configurations have  $n = 4$  seeds; Haiku remains  $n = 1$ . **(L5) Generality probe** currently re-uses the hidden-test inputs; a separate held-out set is in progress. **(L6) Provider coverage is Claude-only.** GPT-4o produced degenerate results in initial runs (a harness  $\times$  function-calling-format interaction not yet root-caused); a non-Claude evaluation is open. **(L7) Domain B** has only 1 session; composability  $\gamma$  is also under-sampled (1 compose task per session). **(L8) The robustness metric** clusters near 0.5 because most LLM-generated tools wrap themselves in try/except; a richer adversarial suite that distinguishes graceful-degradation from semantic-correctness on edge cases would improve discrimination. **(L9) Verifier coverage.** 51/99 tasks currently carry deterministic verifiers (expected mappings or verify predicates); the remaining tasks — concentrated in the API-orchestration session — rely on format-level checks, and the harness now fails closed on any task lacking a verifier. Because the reported TC levels

predate this policy, the robust claims are the cross-system TC ranking/clustering and the task-completion/tool-correctness gap, not the absolute TC level; extending deterministic verification to all 99 tasks and re-running affected configurations is the highest-priority hardening item, tracked by `scripts/audit_tasks.py`.

## 7 Conclusion

Tool-evolving agents create a new evaluation object: the evolving tool library. EVOLVETOOL-BENCH gives researchers a way to diagnose whether that library is correct, reusable, composable, auditable, and regression-free. Three empirical findings motivate the framework.

*Task completion alone is insufficient.* TC clusters in a narrow 63–72% band across six protocols while library health spans a much wider range; we detect no LH–TC relationship ( $\rho = -0.03$ , 95% CI  $[-0.15, 0.09]$ , 180 sessions) and the two dimensions should be reported jointly.

*Reuse decomposition is essential.* The highest-reuse system propagates defects 38.9% of the time. Reporting raw reuse without the correct/incorrect decomposition is misleading for any benchmark of self-evolving agents.

*Capability-aligned conformance tests reveal the specification gap.* Per-tool correctness under hidden tests is near zero (0–6%) despite TC > 60%. Capability-aligned tests provide a signal that self-generated validation cannot.

### Reusable Evaluation Practices

EVOLVETOOL-BENCH is intended less as a fixed leaderboard than as a set of reusable evaluation practices for tool-evolving agents. We recommend that future benchmarks:

- (1) Separate task success from generated-artifact conformance;
- (2) Report correct and incorrect reuse rather than raw reuse alone;
- (3) Include explicit regression tasks after library growth;
- (4) Preserve generated artifacts and source hashes;
- (5) Publish per-task traces sufficient to reproduce decisions;
- (6) Evaluate library-management policies (promotion, deduplication, retirement).

EVOLVETOOL-BENCH is released as an open evaluation framework for any tool- or skill-generating system.<sup>1</sup>

## References

- [Alzubi et al.(2026)] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. 2026. EvoSkill: Automated Skill Discovery for Multi-Agent Systems. *arXiv preprint arXiv:2603.02766* (2026).
- [Backlund and Petersson(2025)] Axel Backlund and Lukas Petersson. 2025. Vending-Bench: A Benchmark for Long-Term Coherence of Autonomous Agents. *arXiv preprint arXiv:2502.15840* (2025).

<sup>1</sup>Code and data will be released upon acceptance; an anonymized snapshot is provided in supplemental material.

- [Bellur et al.(2024)] Abhiram Bellur, Razan Alghamdi, Kidus Workneh, and Joseph Izraelevitz. 2024. Leroy: Library Learning for Imperative Programming Languages. *arXiv preprint arXiv:2410.06438* (2024).
- [Berlot-Attwell et al.(2026)] Ian Berlot-Attwell, Tobias Sesterhenn, Frank Rudzicz, and Xujie Si. 2026. Is This LLM Library Learning? Evaluation Must Account For Compute and Behaviour. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.
- [Bowers et al.(2023)] Matthew Bowers, Theo X. Olausson, Lionel Wong, Gabriel Grand, Joshua B. Tenenbaum, Kevin Ellis, and Armando Solar-Lezama. 2023. Top-Down Synthesis for Library Learning. *Proceedings of the ACM on Programming Languages (POPL)* 7 (2023). doi:10.1145/3571234
- [Cai et al.(2023)] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2023. Large Language Models as Tool Makers. *arXiv preprint arXiv:2305.17126* (2023).
- [Cao et al.(2023)] David Cao, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. 2023. babble: Learning Better Abstractions with E-Graphs and Anti-unification. *Proceedings of the ACM on Programming Languages (POPL)* 7 (2023). doi:10.1145/3571207 arXiv:2212.04596.
- [Ellis et al.(2021)] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. 2021. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. doi:10.1145/3453483.3454080 arXiv:2006.08381.
- [Li et al.(2026)] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. 2026. Skills-Bench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).
- [Liu et al.(2024)] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR)*. arXiv:2308.03688.
- [Nath et al.(2025)] Vaskar Nath, Pranav Raja, Claire Yoon, and Sean Hendryx. 2025. ToolComp: A Multi-Tool Reasoning & Process Supervision Benchmark. *arXiv preprint arXiv:2501.01290* (2025).
- [Qian et al.(2023)] Cheng Qian, Chi Han, Yi R. Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.
- [Qin et al.(2023)] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789* (2023).
- [Singhal et al.(2024)] Manav Singhal, Tushar Aggarwal, Abhijeet Awasthi, Nagarajan Natarajan, and Aditya Kanade. 2024. NoFunEval: Funny How Code LMs Falter on Requirements Beyond Functional Correctness. *arXiv preprint arXiv:2401.15963* (2024).
- [Wang et al.(2023)] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Advances in Neural Information Processing Systems* (2023). arXiv:2305.16291.
- [Wölflein et al.(2025)] Georg Wölflein, Dyke Ferber, Daniel Truhn, Ognjen Arandjelović, and Jakob Nikolas Kather. 2025. LLM Agents Making Agent Tools. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2502.11705.
- [Xia et al.(2026)] Bowei Xia, Mengkang Hu, Shijian Wang, Jiarui Jin, Wenxiang Jiao, Yuan Lu, Kexin Li, and Ping Luo. 2026. Tool-Genesis: A Task-Driven Tool Creation Benchmark for Self-Evolving Language Agents. *arXiv preprint arXiv:2603.05578* (2026).
- [Yuan et al.(2023)] Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R. Fung, Hao Peng, and Heng Ji. 2023. CRAFT: Customizing LLMs by Creating and Retrieving from Specialized Toolsets. *arXiv preprint arXiv:2309.17428* (2023).
- [Zenkner et al.(2024)] Janis Zenkner, Lukas Dierkes, Tobias Sesterhenn, and Christian Bartelt. 2024. AbstractBeam: Enhancing Bottom-Up Program Synthesis using Library Learning. *arXiv preprint arXiv:2405.17514* (2024).
- [Zhang et al.(2026)] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. 2026. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. *arXiv preprint arXiv:2604.01687* (2026).